(continued)

in. Write a new view `venues_restricted()` that is only visible by those users associated with a `Venue` object and displays a list of all venues. Hint: you can take advantage of other views in your code; they are just functions.

To implement `venues_restricted()`, you need two new functions: one for the view itself and one to pass the `@user_passes_test` decorator. As the view does the same thing as the `venues()` view, it can call it directly, saving you some code. This is a little nonsensical, as you wouldn't provide both a restricted and nonrestricted version of the same thing—your users could just use the nonrestricted version—but the exercise does demonstrate how to use `@user_passes_test` to implement more restrictions than a required login. The following listing shows both the new view and the function passed to the decorator.

Listing E.22 Restricting a view using `@user_passes_test`

```
# RiffMates/bands/views.py
from django.contrib.auth.decorators import user_passes_test
from django.shortcuts import render
...

def has_venue(user):
    try:
        return user.userprofile.venues_controlled.count() > 0
    except AttributeError:
        return False


@user_passes_test(has_venue)
def venues_restricted(request):
    return venues(request)
```

This function is passed as a reference to @user_passes_test; it should return True to indicate the user is allowed in or False otherwise.

Check how many venues the user controls; if it is one or more, allow them in.

Anonymous users have no profile, which triggers an AttributeError: users that are not logged in cannot control a venue, so they fail the test.

This decorator restricts access to the view based on the result from the function its argument references.

Views are functions; you can use one view to get information from another, like any other function.

For this view to work, it needs a route; for brevity, this has been left out. See the exercise in the following listing for an example.

Exercise 2

The auth system emits signals on login events. Capture the `django.contrib.auth.signals.user_login_failed` signal, and print a message displaying the username and path of the login attempt. See https://docs.djangoproject.com/en/dev/ref/contrib/auth/#topics-auth-signals for details on the arguments passed when receiving this signal. (Note that in production, you should use logging rather than `print()`—see appendix B for more details.)